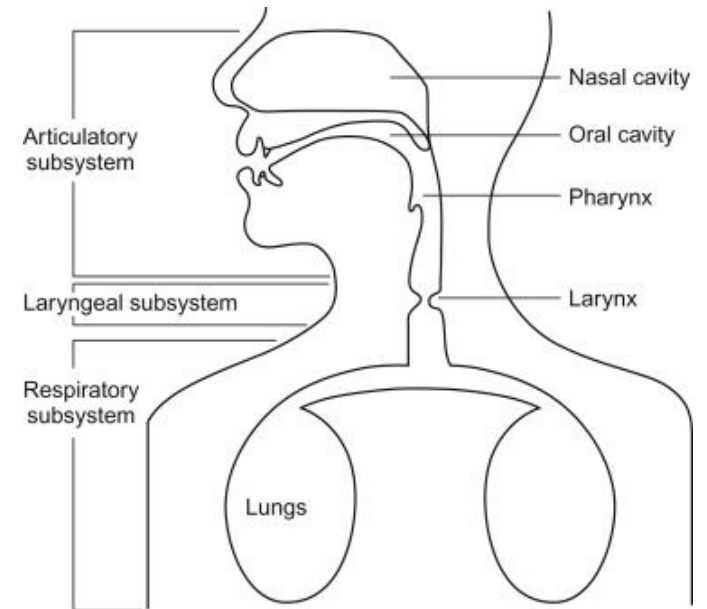


Speech Processing-2

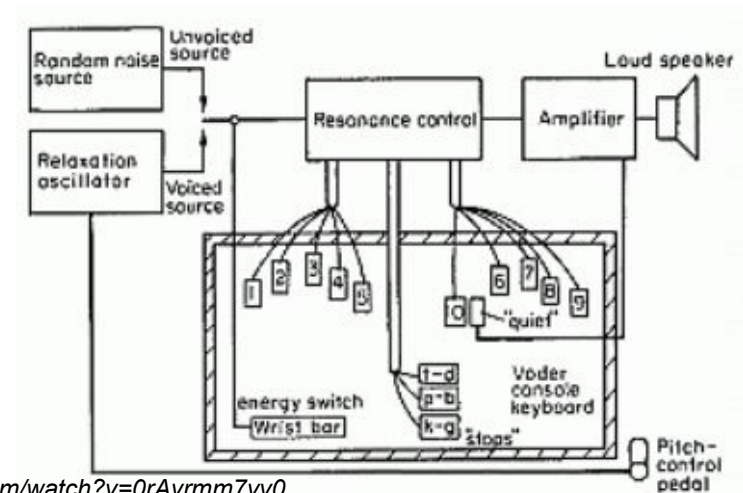
Dr. Shaik Mohammad Rafi B
UG3 S2026
Indian Institute of Information Technology Sri City

Speech Synthesis

- Let machines talk like humans
- Early approaches tried to build a mechanical model of the human vocal tract.
- Wolfgang von Kempelen's Speaking Machine (1791): which used bellows for lungs and a reed (clarinets) for vocal folds.
- Homer Dudley's VODER (1939): First device to synthesize speech sounds via electrical means.



https://www.youtube.com/watch?v=k_YUB_S6Gpo



<https://www.youtube.com/watch?v=0rAyrmm7vv0>

Drawbacks

- Cannot reach high simulation accuracy
- Difficult to build
- Difficult to control (manual)
- Difficult to make adjustment

Shift in paradigm

- Moving the focus from speaking to speech, from action to result. Mechanical systems and signal processing are complex to handle.
- A model of the vocal tract and a model of the speech production process are involved.
- Move closer to result: Model with data

TTS system development

Goal of a TTS system: Produce a natural-sounding high quality speech waveform for a given word sequence.

TTS systems are typically divided into two parts:

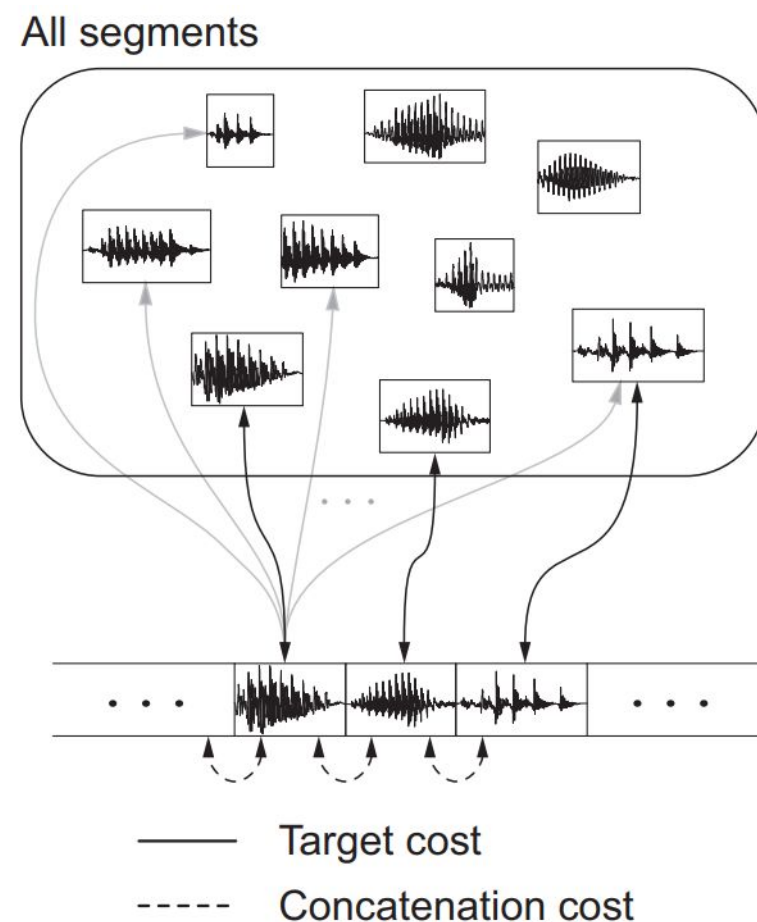
- Linguistic specification
- Waveform generation

Two prominent instances of corpus-based TTS

1. Unit selection and concatenation
2. Statistical parametric speech synthesis

Unit selection and concatenation

- Synthesize new sentences by selecting sub-word units from a database of speech.
- Given text, the system looks for the best sequence of units in its database. It calculates two types of "costs":
 - Target Cost: How well does the unit in the database match the requirements of the target sentence (e.g., correct phoneme, stress, and pitch)?
 - Concatenation Cost: How well will two units sound when put together? The system looks at spectral and pitch continuity.
- Stitches the best path audio segments together



Text to Speech using statistical parametric models (HMMs)

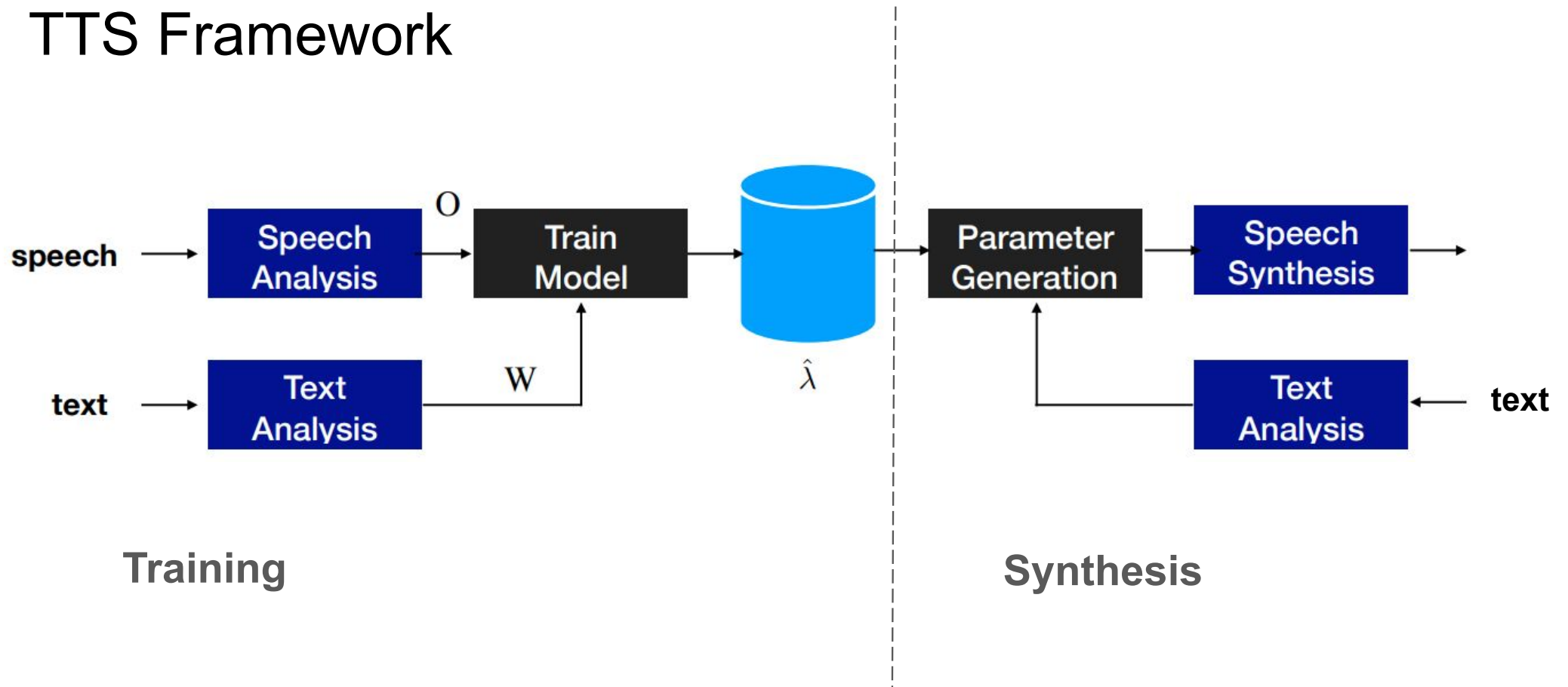
TTS is the process of converting written graphemes into audible speech.

The Three steps in TTS:

1. Text Analysis: Converting raw text into phonetic and linguistic labels.
2. Acoustic Modeling (HMM): Mapping labels to statistical sound parameters (MFCCs, F₀).
3. Waveform Synthesis: Turning parameters into a sound wave using a Vocoder.

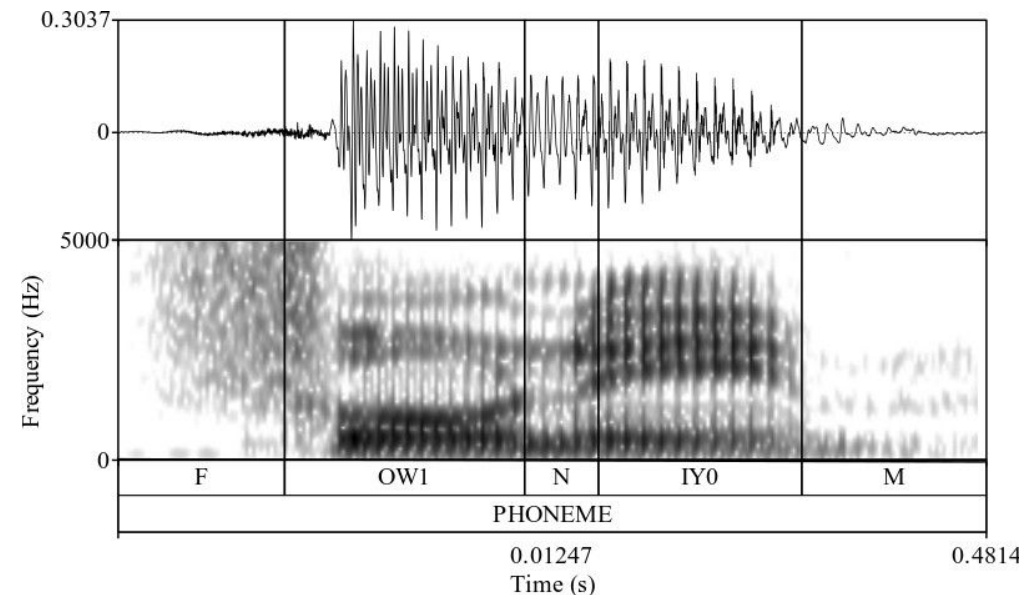
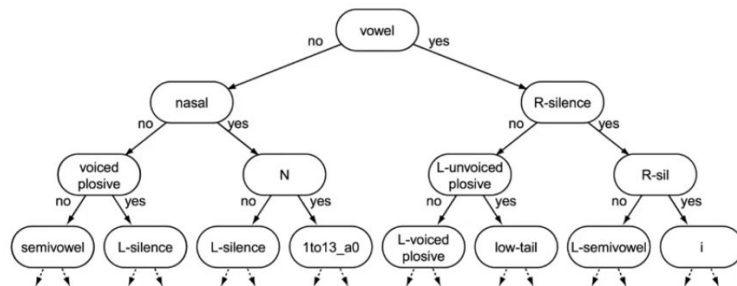
HMM-TTS is *parametric*—it doesn't play back recorded clips; it models the sound based on math.

TTS Framework



Text processing

1. Normalization: Converting "1995" to "nineteen ninety-five."
2. G2P (Grapheme-to-Phoneme): Using a dictionary to turn "Lead" into /l/ /i:/ /d/.
3. Part-of-Speech (POS) Tagging: Identifying if a word is a Noun or Verb to determine correct stress.
4. Prosody Prediction: Predicting where pauses should go and how the pitch should rise or fall.
5. Get Forced Alignment of the transcriptions (duration of phonemes, from a pre-trained Speech Recognizer).
6. Generate context labels
 - a. Phonetic neighbors.
 - b. Syllable stress and position.
 - c. Word position in the sentence.
7. Clustering with decision trees
 - a. grouping billions of possible context labels.

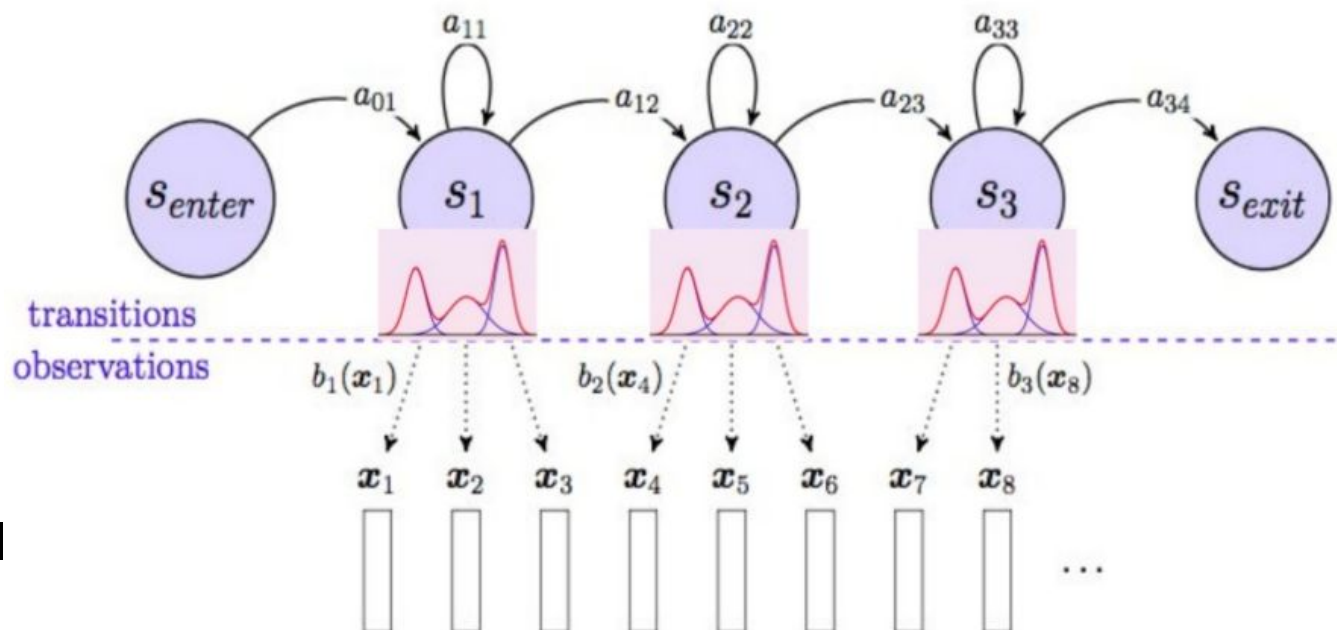


Hidden Markov Models: Acoustic modelling

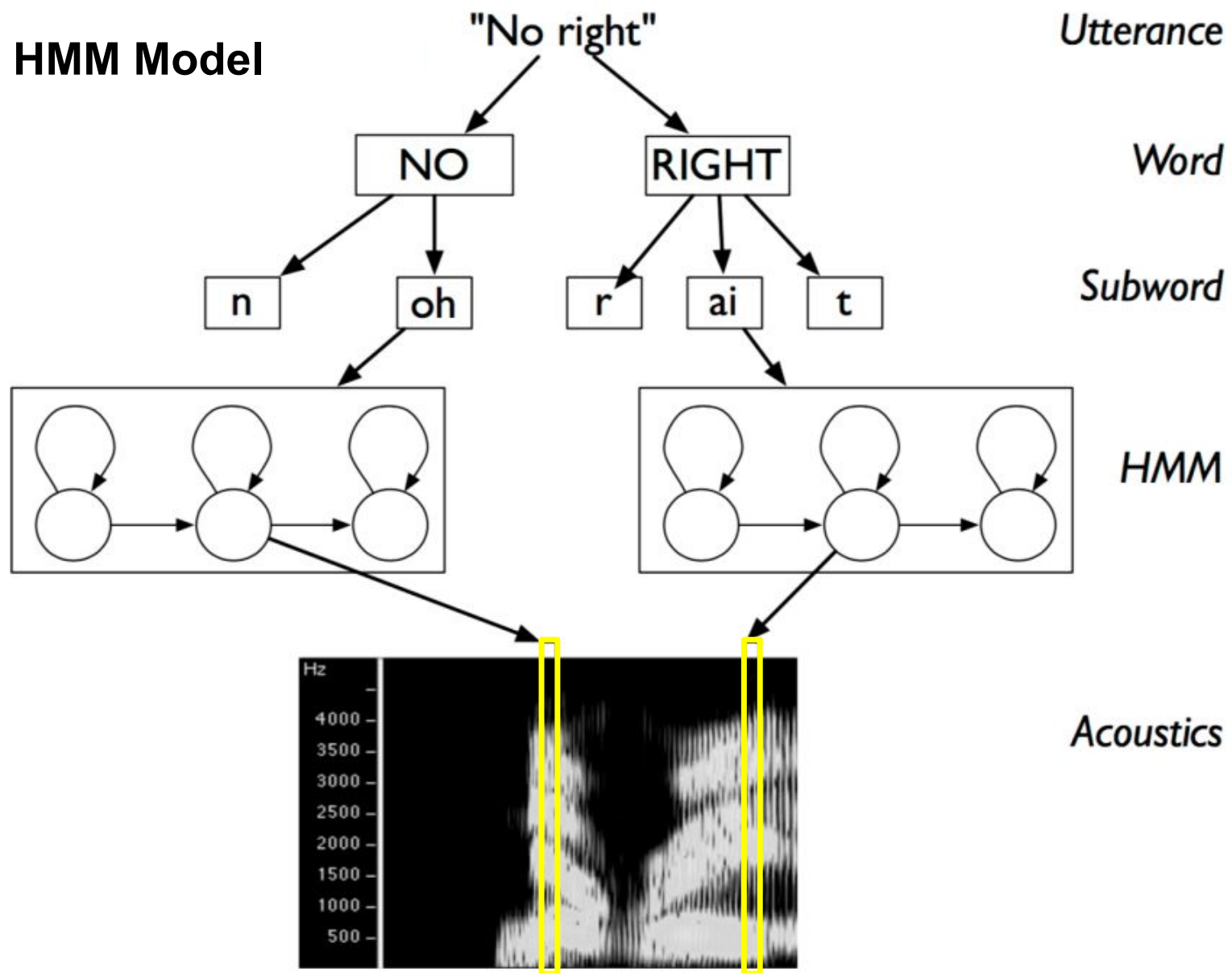
Every phoneme is assumed as 5 state transition and generating few MFCC vectors.

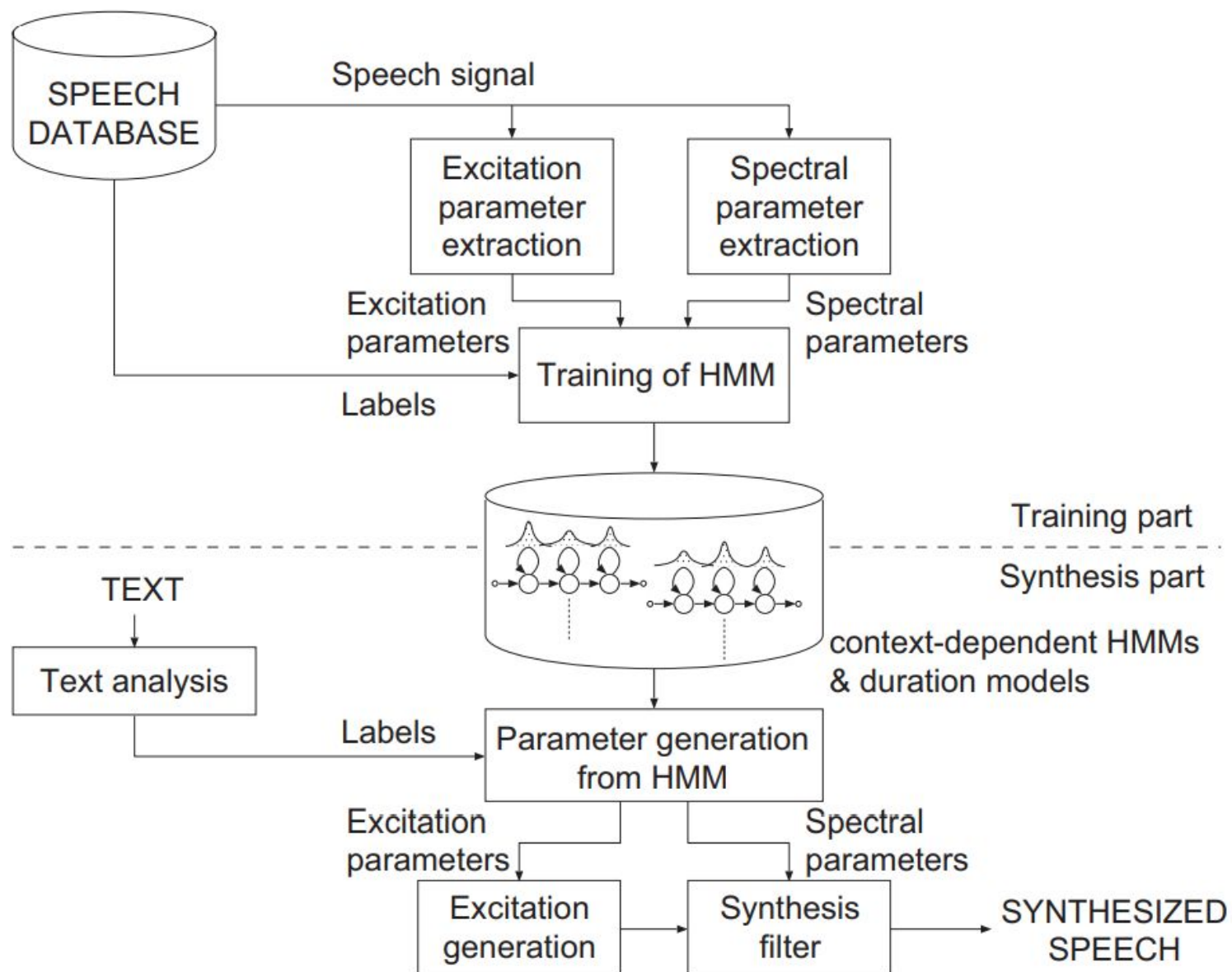
Each phoneme is described by

- 5x5 transition probability matrix $\{a_{ij}\}$
- Emission probability model as 64 component GMM (64 mean vectors and 64 variance vectors).



HMM Model





Duration Modelling

- In HMM-based speech synthesis, the model only knows what the spectrum should look like, but not how long a person stays in a specific state.
- Without a duration model, the synthesis would sound either rushed or infinitely sustained.

Duration modelling

1. Direct specification
 - 'iy': [4, 6, 12, 6, 4], # Vowels have longer middle states
 - 'sh': [8, 10, 10, 10, 8], # Fricatives are relatively consistent
 - 'hh': [3, 3, 3, 3, 3], # Aspirations are short
 - 'sil': [20, 20, 20, 20, 20]
 - Default: [5,5,5,5,5]
2. Based on Self-transition probability a_{ii} (duration= $1/a_{ii}$)
3. Duration of each state as a Gaussian distribution during training.

TTS Inference with Given Text

Generating the Sentence Model

- Step 1: Convert input text into a sequence of Full-Context Labels.
- Step 2: Traverse the Decision Trees to find the best-matching HMM states for each label.
- Step 3: Sample MFCC vectors for each label from HMM.
- Step 3: Concatenate these MFCCs to form a Sentence HMM.
- Duration: Use the duration model to decide exactly how many frames each state (MFCCs) will occupy.
- Use a Vocoder (MFCC to waveform converter) to get synthesized waveform.

An introduction to HMM based TTS by J. Yamagishi

<https://scispace.com/pdf/an-introduction-to-hmm-based-speech-synthesis-3p7z3xdjs3.pdf>

Speech Recognition

Transform a spoken utterance into a sequence of tokens (words, syllables, phonemes, characters)

Applications of ASR:

- Speech understanding
- Spoken translation
- Audio information retrieval

ASR handles multiple variabilities: Speaker style, accents, room acoustics, microphone properties, etc.

Cascaded ASR System

1. Acoustic Model (AM)

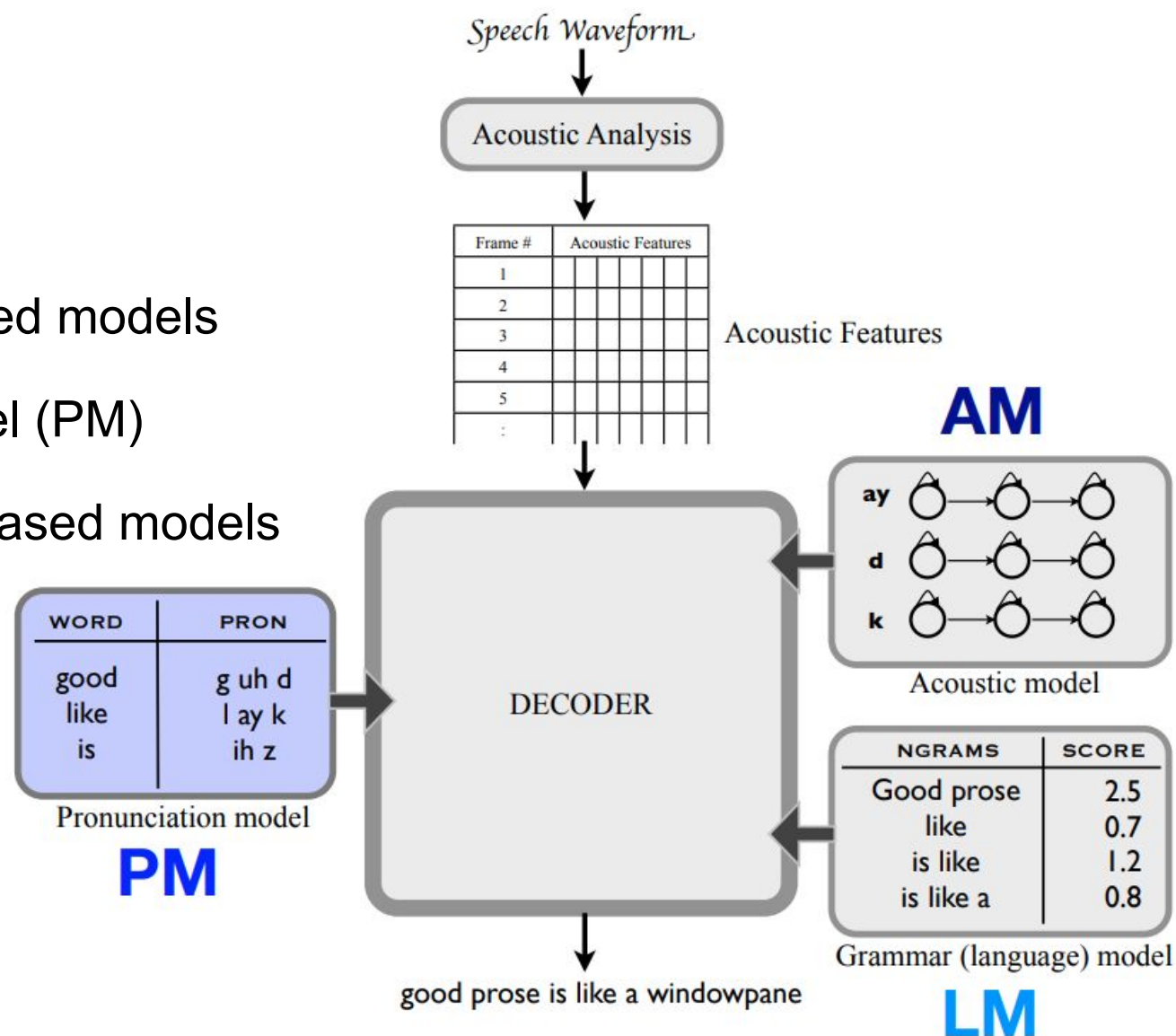
HMMs, DNN and RNN-based models

2. Pronunciation (Lexicon) Model (PM)

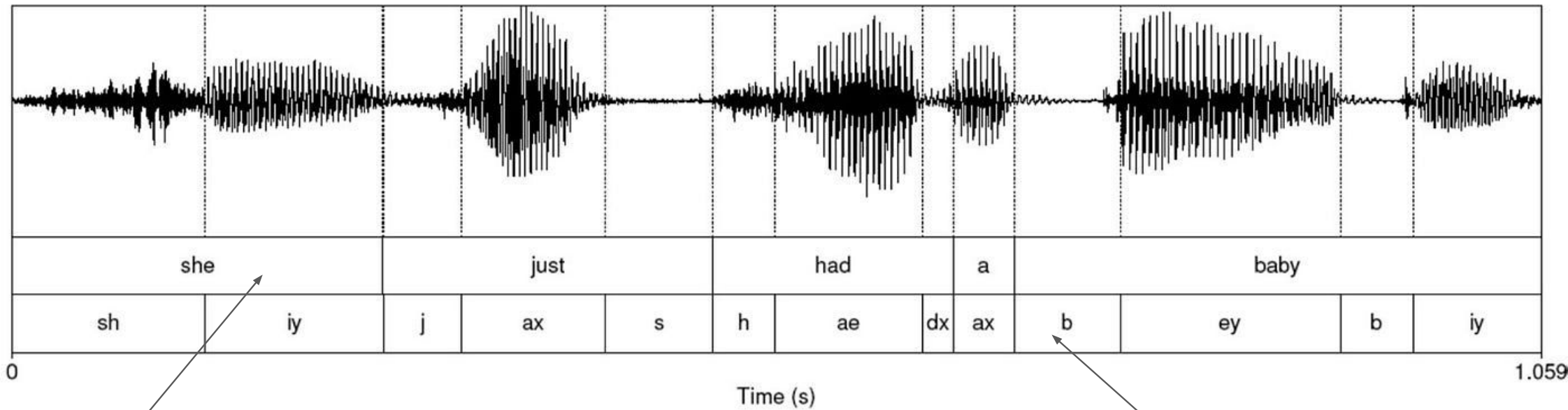
Phoneme and Grapheme-based models

3. Language Model (LM)

Ngram models (text)



Speech to text (stt)



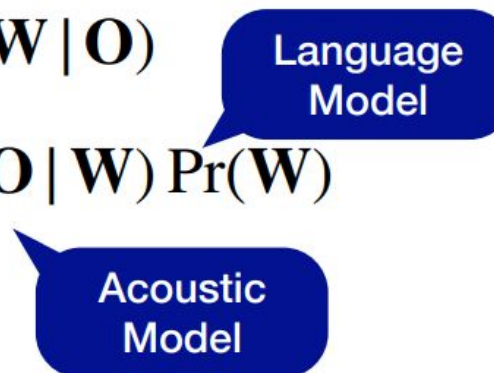
Language model : $p(\text{sentence})$

Pronunciation model

Statistical Speech Recognition

Let $\mathbf{O}$ be a sequence of acoustic features corresponding to a speech signal. That is, $\mathbf{O} = \{O_1, \dots, O_T\}$, where $O_i \in \mathbb{R}^d$ refers to a d-dimensional acoustic feature vector and T is the length of the sequence.

Let $\mathbf{W}$ denote a word sequence. An ASR decoder solves the foll. problem:

$$\begin{aligned}\mathbf{W}^* &= \arg \max_W \Pr(\mathbf{W} | \mathbf{O}) \\ &= \arg \max_W \Pr(\mathbf{O} | \mathbf{W}) \Pr(\mathbf{W})\end{aligned}$$


The diagram illustrates the decomposition of the ASR problem. The first line of the equation, $\Pr(\mathbf{W} | \mathbf{O})$, is associated with a blue speech bubble labeled "Language Model". The second line, $\Pr(\mathbf{O} | \mathbf{W}) \Pr(\mathbf{W})$, is associated with a blue speech bubble labeled "Acoustic Model".

HMM parameters

$Q = q_1 q_2 \dots q_N$	a set of N states
$A = a_{11} \dots a_{ij} \dots a_{NN}$	a transition probability matrix A , each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^N a_{ij} = 1 \quad \forall i$
$O = o_1 o_2 \dots o_T$	a sequence of T observations , each one drawn from a vocabulary $V = v_1, v_2, \dots, v_V$
$B = b_i(o_t)$	a sequence of observation likelihoods , also called emission probabilities , each expressing the probability of an observation o_t being generated from a state i
$\pi = \pi_1, \pi_2, \dots, \pi_N$	an initial probability distribution over states. π_i is the probability that the Markov chain will start in state i . Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^n \pi_i = 1$

How ASR works

- A: speech utterance
- O_A : acoustic features corresponding to the utterance A

$$W^* = \arg \max_W \Pr(O_A|W) \Pr(W)$$

- Return the word sequence that jointly assigns the highest probability to O_A
- How do we estimate $\Pr(O_A|W)$ and $\Pr(W)$?
- How do we decode?

Acoustic model

$$W^* = \arg \max_W \Pr(O_A|W) \Pr(W)$$

$$\Pr(O_A|W) = \sum_Q \Pr(O_A, Q|W)$$

State + word
sequence

$$= \sum_{q_1^T, w_1^N} \prod_{t=1}^T \Pr(O_t | O_1^{t-1}, q_1^t, w_1^N) \Pr(q_t | q_1^{t-1}, w_1^N)$$

First-order HMM
assumptions

$$\approx \sum_{q_1^T, w_1^N} \prod_{t=1}^T \Pr(O_t | q_t, w_1^N) \Pr(q_t | q_{t-1}, w_1^N)$$

Viterbi approximation

$$\approx \max_{q_1^T, w_1^N} \prod_{t=1}^T \Pr(O_t | q_t, w_1^N) \Pr(q_t | q_{t-1}, w_1^N)$$

Acoustic Model

$$\Pr(O_A|W) = \max_{q_1^T, w_1^N} \prod_{t=1}^T \Pr(O_t|q_t, w_1^N) \Pr(q_t|q_{t-1}, w_1^N)$$

Transition probabilities

Emission probabilities

Derived from a DNN or TDNN model

Modeled using a mixture of Gaussians

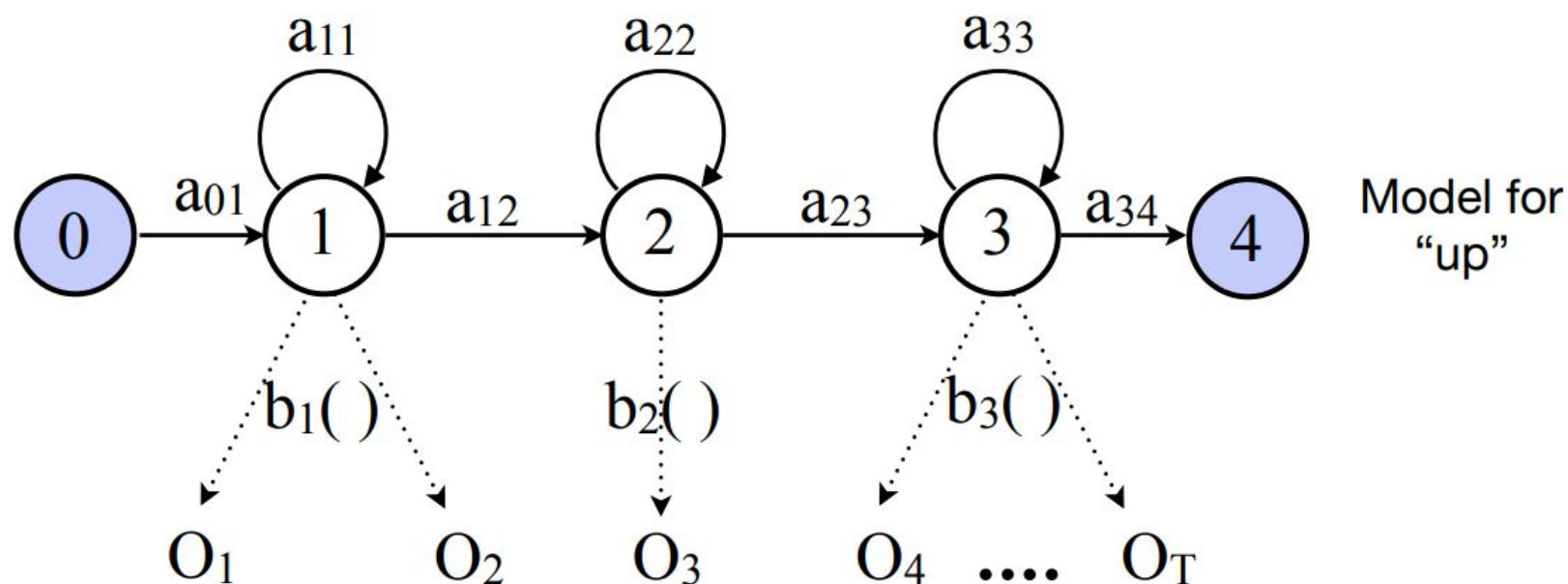
$$\Pr(O|q; w_1^N) = \sum_{\ell=1}^{L_q} c_{q\ell} \mathcal{N}(O|\mu_{q\ell}, \Sigma_{q\ell}; w_1^N)$$

$$\Pr(O|q; w_1^N) \propto \frac{\Pr(q|O; w_1^N)}{\Pr(q)}$$

Isolated word ASR

- Task: Recognize utterances which consist of speakers saying either “up” or “down” or “left” or “right” per recording.
- Vocabulary: Four words, “up”, “down”, “left”, “right”
- Data splits
 - Training data: 30 utterances
 - Test data: 20 utterances
- Acoustic model: Let's parameterize $\Pr_{\theta}(\mathbf{O} \mid \mathbf{W})$ using a Markov model with parameters θ .

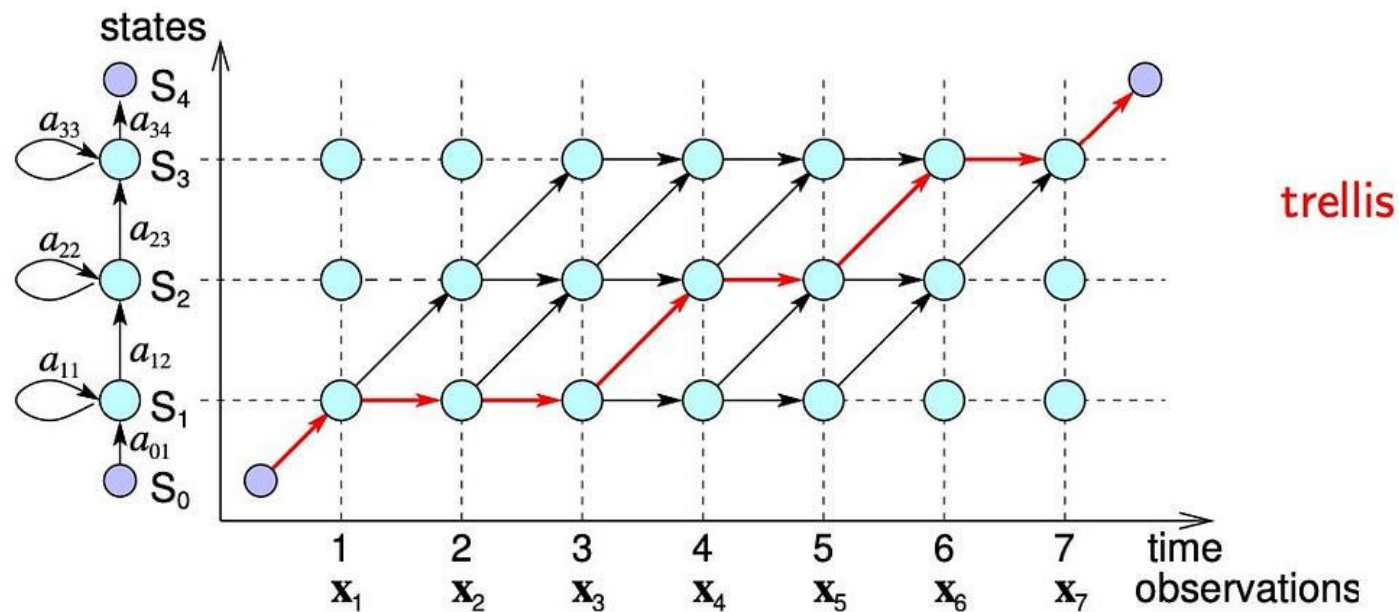
Isolated word model using HMMs



$a_{ij} \rightarrow$ Transition probabilities going from state i to state j

$b_j(O_i) \rightarrow$ Probability of generating O_i from state j

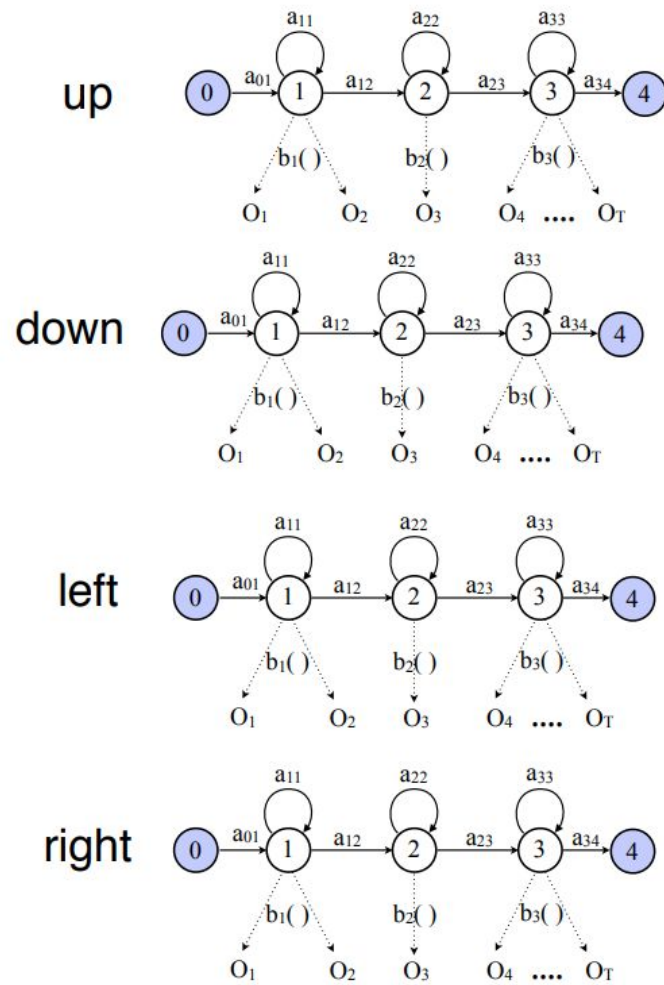
Decoding



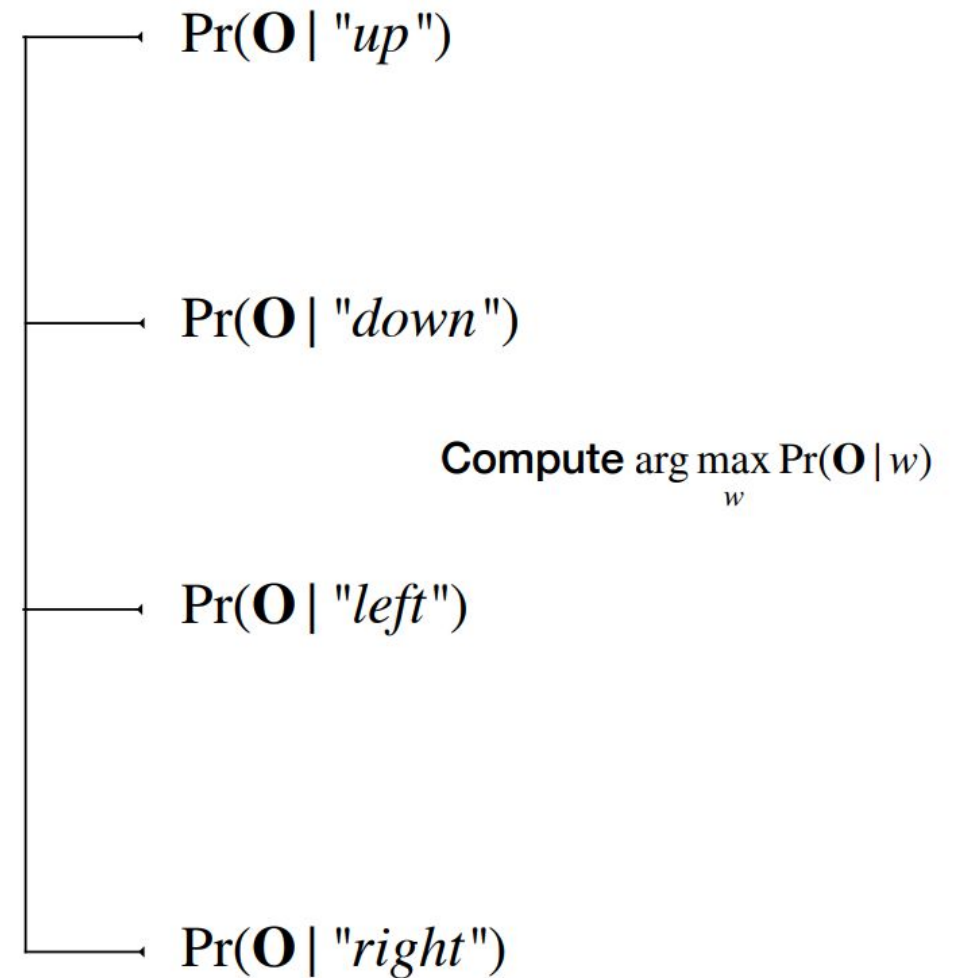
$$p(\mathbf{X}, \text{path}_\ell | \lambda) = p(\mathbf{X} | \text{path}_\ell, \lambda) P(\text{path}_\ell | \lambda)$$

$$\text{likelihood: } \sum_{\{\text{path}_\ell\}} p(\mathbf{X}, \text{path}_\ell | \lambda)$$

$$\text{decode: } \max_{\text{path}_\ell} p(\mathbf{X}, \text{path}_\ell | \lambda)$$



acoustic
features
 $\mathbf{O}$



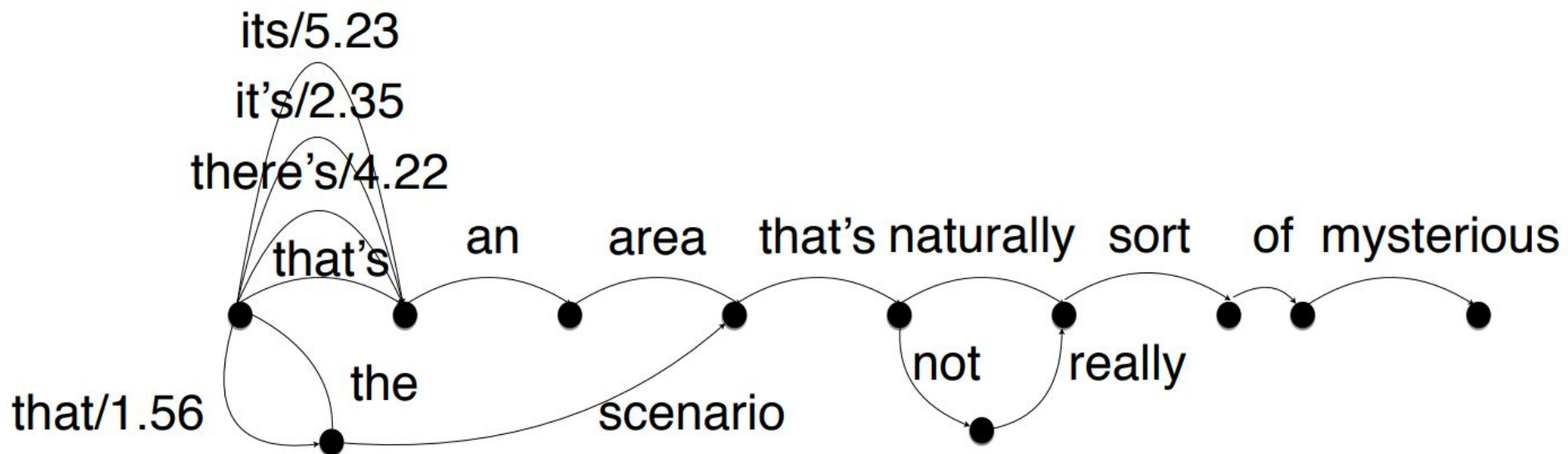
Trained HMM models

Language Model

$$W^* = \arg \max_W \Pr(O_A|W) \Pr(W)$$

$$\begin{aligned} \Pr(W) &= \Pr(w_1, w_2, \dots, w_N) \\ &= \Pr(w_1) \dots \Pr(w_N | w_{N-m+1}^{N-1}) \end{aligned}$$

m-gram language model



Rank	Path	AM logprob	LM logprob
1.	it's an area that's naturally sort of mysterious	-7193.53	-20.25
2.	that's an area that's naturally sort of mysterious	-7192.28	-21.11
3.	it's an area that's not really sort of mysterious	-7221.68	-18.91
4.	that scenario that's naturally sort of mysterious	-7189.19	-22.08
5.	there's an area that's naturally sort of mysterious	-7198.35	-21.34
6.	that's an area that's not really sort of mysterious	-7220.44	-19.77
7.	the scenario that's naturally sort of mysterious	-7205.42	-21.50
8.	so it's an area that's naturally sort of mysterious	-7195.92	-21.71
9.	that scenario that's not really sort of mysterious	-7217.34	-20.70
10.	there's an area that's not really sort of mysterious	-7226.51	-20.01

Speaker Recognition

Ask of recognizing a speaker from his/her voice

- Identification: Whether an unknown speaker matches one in the list?
- Verification: Whether an unknown speaker is who she claims to be?

Applications of voice biometric

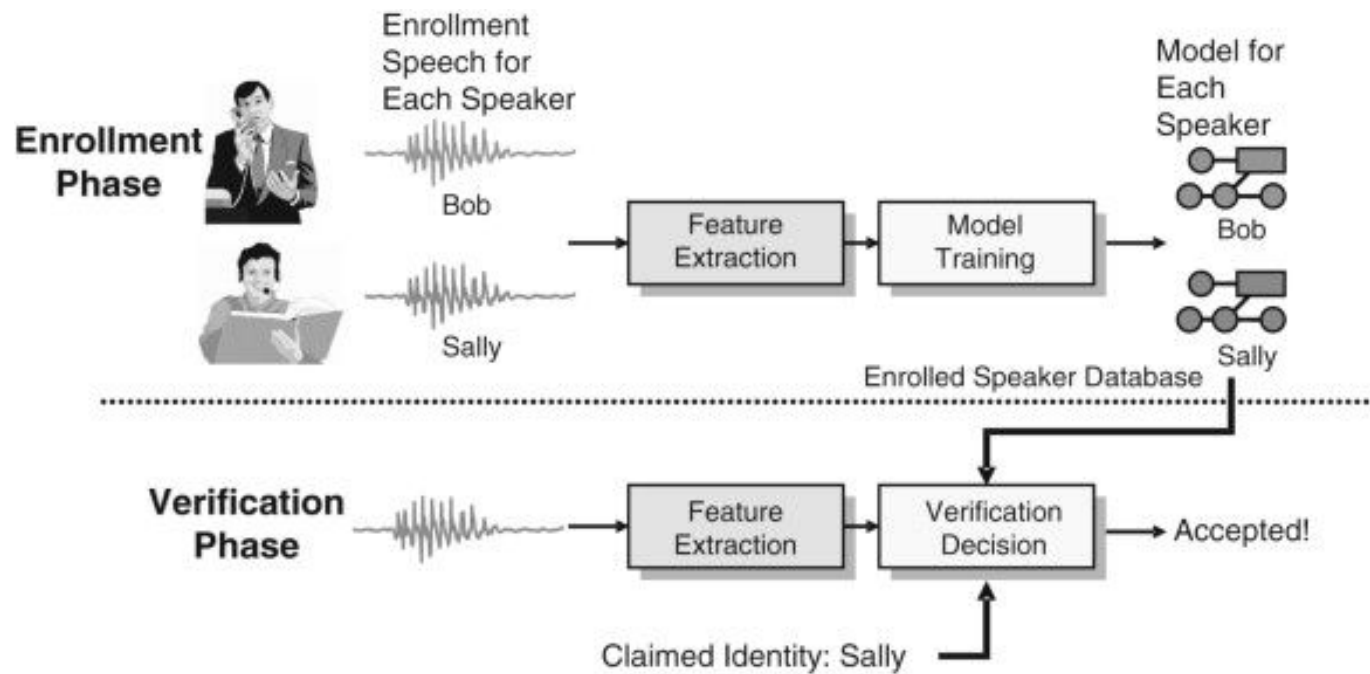
- Remotely recognizable & contactless
- Authenticating mobile devices, voice mails, telebanking etc.
- Forensic applications

Not an easy task: Humans make 23% errors on unfamiliar voices

Speaker-specific information in the speech signal

- Anatomical differences: VT size & shape, vocal-fold thickness, pitch..
- Learned speaking habits: dialect, prosody, speaking rate, disfluencies

Speaker Verification



Types

Text-independent speaker recognition

- Text content of enrollment and test utterances need not match
- Recognition system does not know text spoken at test time
- Offers a flexible system, but difficult to realize
- Useful in forensic applications
- Statistical pattern matching techniques are used

Text-dependent speaker recognition

- Enrollment and test utterances should be of same text
- Recognition system knows the text spoken at test time
- Knowledge of known text improves the performance
- Applications involving cooperative users (banking, mobile phones)
- Template matching techniques, like DTW, are used

Text independent SV

- As text content is different, sequence feature vectors do not match
- The enrollment and test utterances could be of different length
- Estimate the pdf of features from enrollment utterance
- Evaluate probability of drawing test data from the estimated pdf
- Assign speaker- k if $p(\text{test data}/\lambda_k)$ is the highest

Estimating pdf by Maximum likelihood

- Consider data $X = \{x_1, x_2, \dots, x_N\}$ drawn from unknown distribution
- Underlying distribution $p(x_n)$ be approximated by a parametric form
 - Gaussian assumption: $p(x_n) \sim \mathcal{N}(x_n/\mu, \sigma^2)$
 - Laplacian assumption: $p(x_n) \sim \mathcal{L}(x_n/\mu, b)$
- Assuming i.i.d, the likelihood function can be written as

$$\begin{aligned} p(X/\mu, \sigma^2) &= \prod_{n=1}^N p(x_n/\mu, \sigma^2) \\ &= \prod_{n=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_n - \mu)^2}{2\sigma^2}\right) \end{aligned}$$

- Estimate μ and σ to maximize the likelihood function, or eqv.

$$J(\mu, \sigma^2) = \log p(X/\mu, \sigma^2)$$

- μ and σ can be determined by equating the derivatives to zero

$$\hat{\mu} = \frac{1}{N} \sum_{n=1}^N x_n \quad \hat{\sigma}^2 = \frac{1}{N} \sum_{n=1}^N (x_n - \hat{\mu})^2$$

- Checking for *bias* in estimation: Taking expectation over estimates

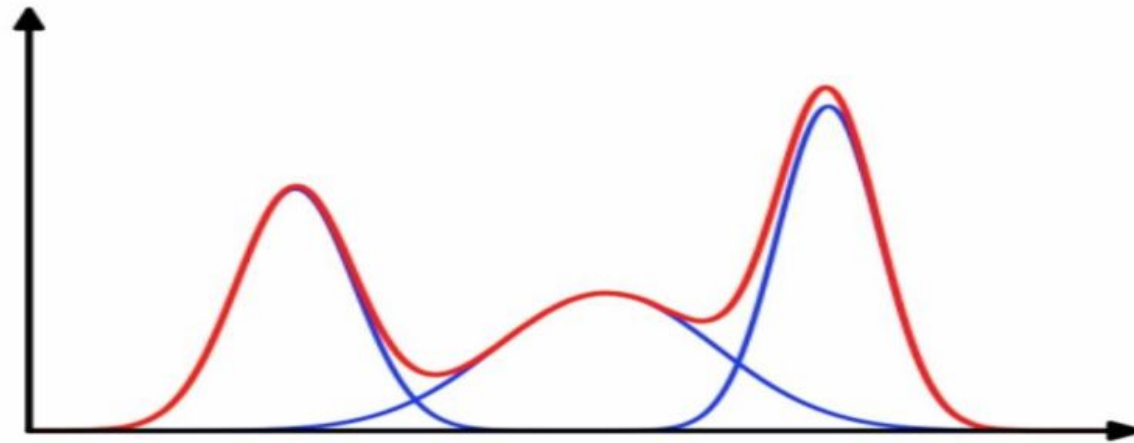
$$\mathbb{E}[\hat{\mu}] = \mathbb{E} \left[\frac{1}{N} \sum_{n=1}^N x_n \right] = \frac{1}{N} \sum_{n=1}^N \mathbb{E}[x_n] = \mu \quad \text{unbiased}$$

$$\mathbb{E}[\hat{\sigma}^2] = \frac{1}{N} \sum_{n=1}^N \mathbb{E}[(x_n - \hat{\mu})^2] = \frac{N-1}{N} \sigma^2 \quad \text{biased}$$

- Variance is underestimated in ML approach. $\hat{\sigma}^2 < \sigma^2$

- The variance estimation can be corrected as $\tilde{\sigma}^2 = \frac{1}{N-1} \sum_{n=1}^N (x_n - \hat{\mu})^2$

Gaussian Mixture model



- PDF can be written as a linear weighted sum of Gaussian distributions
- Linear superposition of Gaussians

$$p(x) = \sum_{m=1}^M \pi_m \mathcal{N}(x/\mu_m, \sigma_m^2)$$

ML formulation for GMM

- Consider data $X = \{x_1, x_2, \dots, x_N\}$ drawn from unknown distribution
- Let the unknown distribution $p(x_n)$ be approximated by a GMM

$$p(x_n) = \sum_{m=1}^M \pi_m \mathcal{N}(x_n / \mu_m, \sigma_m^2)$$

- Assuming i.i.d sampling, the likelihood function can expressed as

$$\mathcal{L}(\theta/X) = P(X/\theta) = \prod_{n=1}^N \sum_{m=1}^M \pi_m \mathcal{N}(x_n / \mu_m, \sigma_m^2)$$

- GMM parameters can be estimated by maximizing the log-likelihood

$$J(\theta) = \sum_{n=1}^N \log \left(\sum_{m=1}^M \pi_m \mathcal{N}(x_n / \mu_m, \sigma_m^2) \right) + \lambda \sum_{m=1}^M \pi_m = 1$$

$$J(\theta) = \sum_{n=1}^N \log \left(\sum_{m=1}^M \pi_m \mathcal{N}(x_n / \mu_m, \sigma_m^2) \right) + \lambda \sum_{m=1}^M \pi_m = 1$$

- Taking partial derivative w.r.t μ_k

$$\begin{aligned} \frac{\partial J(\theta)}{\partial \mu_k} &= \sum_{n=1}^N \frac{1}{\sum_{m=1}^M \pi_m \mathcal{N}(x_n / \mu_m, \sigma_m^2)} \pi_k \frac{\partial}{\partial \mu_k} \mathcal{N}(x_n / \mu_k, \sigma_k^2) \\ &= \sum_{n=1}^N \frac{\pi_k \mathcal{N}(x_n / \mu_k, \sigma_k^2)}{\sum_{m=1}^M \pi_m \mathcal{N}(x_n / \mu_m, \sigma_m^2)} \left(\frac{x_n - \mu_k}{\sigma_k^2} \right) \\ &= \sum_{n=1}^N \gamma_{nk} \left(\frac{x_n - \mu_k}{\sigma_k^2} \right) \end{aligned}$$

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(x_n / \mu_k, \sigma_k^2)}{\sum_{m=1}^M \pi_m \mathcal{N}(x_n / \mu_m, \sigma_m^2)}$$

- γ_{nk} : Responsibility of k^{th} Gaussain in generating n^{th} datapoint
- All components should collectively share the responsibility

$$\sum_{k=1}^M \gamma_{nk} = 1$$

- Effective number of points generated by k^{th} component

$$\sum_{n=1}^N \gamma_{nk} = N_k$$

- Estimated mean of the k^{th} component

$$\hat{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} x_n \quad k = 1, 2, \dots, M$$

- Estimated variance of the k^{th} component

$$\hat{\sigma}_k^2 = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (x_n - \hat{\mu}_k)^2 \quad k = 1, 2, \dots, M$$

- Estimated weight of the k^{th} component (prior)

$$\hat{\pi}_k = \frac{N_k}{N} \quad k = 1, 2, \dots, M$$

EM algorithm for GMM

- Given data $X = \{x_1, x_2, \dots, x_N\}$
- **Initialization:** Choose the number of mixtures M
 - Randomly choose means (μ_k) , variances (σ_k^2) and mixture weights (π_k)
 - Evaluate log-likelihood with initial conditions $p(X/\theta)$
- **Expectation step:** Evaluate the responsibilities using the current θ

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(x_n / \mu_k, \sigma_k^2)}{\sum_{m=1}^M \pi_m \mathcal{N}(x_n / \mu_m, \sigma_m^2)} \quad N_k = \sum_{n=1}^N \gamma_{nk}$$

- **Maximization step:** Update parameters with current γ_{nk}

$$\hat{\mu}_k^{new} = \frac{1}{N_k} \quad \hat{\sigma}_k^{new} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (x_n - \hat{\mu}_k)^2 \quad \hat{\pi}_k^{new} = \frac{N_k}{N}$$

Repeat E-step and M-step until convergence

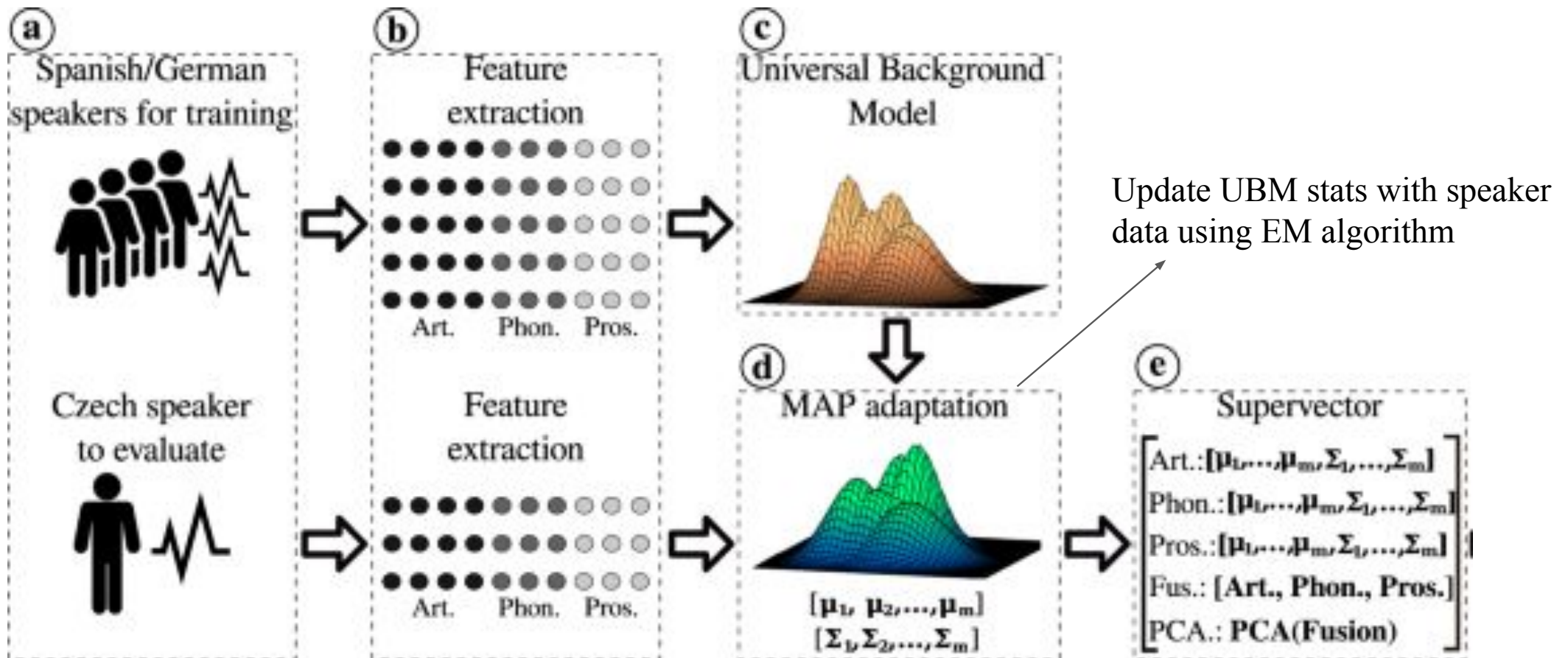
Speaker verification using GMMs

- UBM: Gather all feature vectors from all available speakers in the background dataset.
- Train a GMM for the background dataset to get a Universal Background Model (UBM): $\lambda_{bg} = \{w, \mu, \Sigma\}$
- For given speaker data, extract feature vectors.
- Train Speaker GMM $\Rightarrow \lambda_s = \{w, \mu, \Sigma\}$; using EM algorithm.
- For a given testing utterance, extract features x and compute likelihood ratio

$$\frac{p(x/\lambda_s)}{p(x/\lambda_{bg})} \begin{cases} \geq \tau & \text{Accept} \\ < \tau & \text{Reject} \end{cases}$$

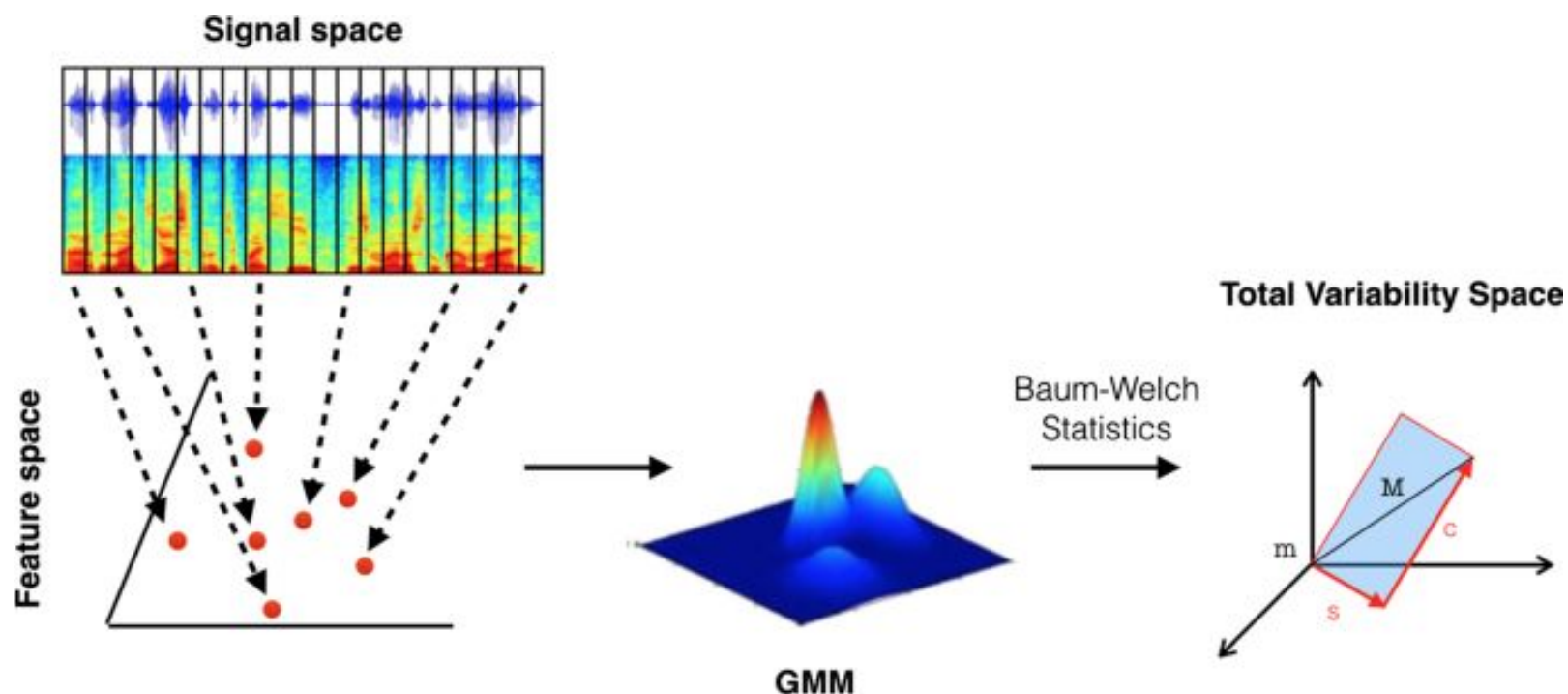
Here T is threshold tuned with respect to training dataset.

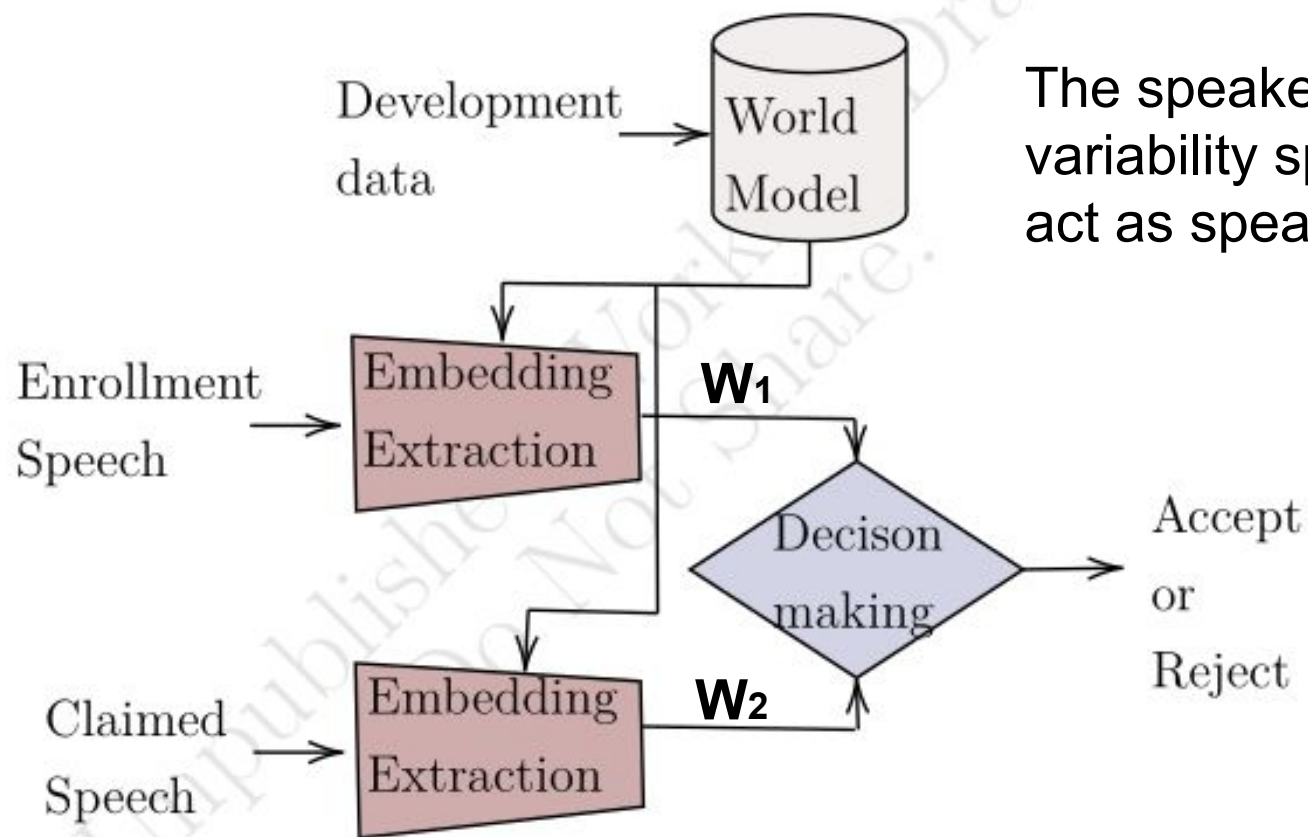
Speaker modeling using Universal Background Model



SPeaker verification by speaker embeddings

GMM supervector is projected onto a total variability space to get speaker embedding.





The speaker representation from total variability space is called as i-vector act as speaker embedding.

A similarity measure is computed between i-vectors of enrollment and claimed speech for decision making.

$$\cos(\mathbf{w}_1, \mathbf{w}_2) = \frac{\mathbf{w}_1 \cdot \mathbf{w}_2}{\|\mathbf{w}_1\| \|\mathbf{w}_2\|}$$

Speaker verification using Neural Networks

- A time-dilated 1D CNN is used to extract frame level representations
- 1st order and second order stats are computed to get fixed length representations
- These representations are used to classify background speakers using a couple of dense layers.
- For any enrolled utterance, the penultimate layer activations are collected as speaker embedding called as x-vector.

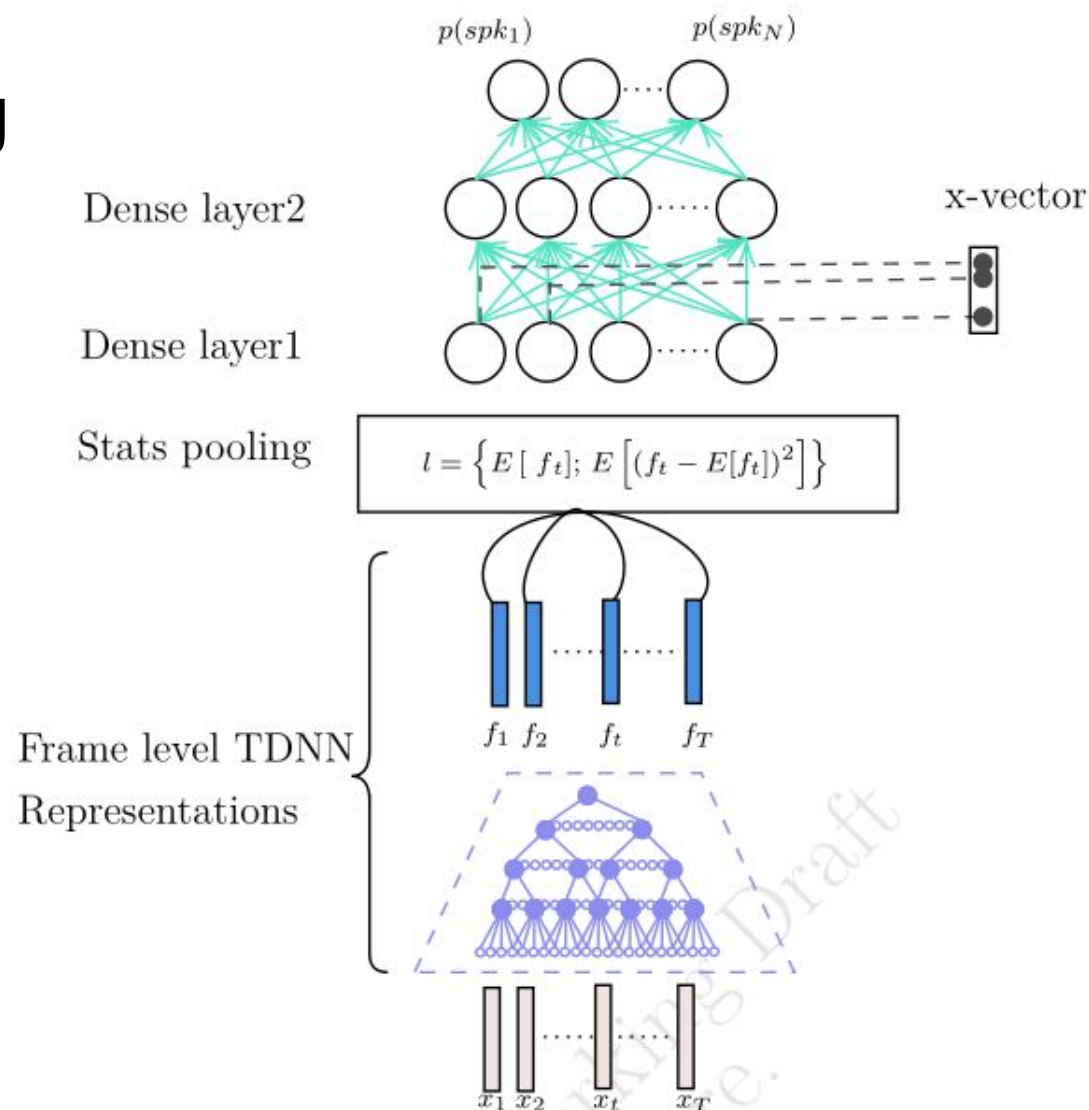


Figure 2.7: x-vector architecture

Speech Prosody: Rhythm, Intonation, and Stress

1. Rhythm: Rhythm is the patterned recurrence of sound units over time. It is determined by how a language distributes the "beats" across syllables.

1. CATS / eat / MICE. (3 syllables)
2. The CATS / will eat / the MICE. (6 syllables)
3. The CATS / will have eaten / the MICE. (9 syllables)

2. Intonation: Intonation is the variation of **pitch** (highness or lowness) while speaking. It doesn't change the dictionary definition of a word, but it changes the *intent* or *emotion* of the sentence.

- **Rising Intonation:** Usually indicates a question or uncertainty.
 - *Example:* "You're coming?" (Pitch goes up at the end).
- **Falling Intonation:** Usually indicates a statement, a command, or completion.
 - *Example:* "I am leaving." (Pitch drops at the end).
- **Sarcasm:** Pitch patterns often become exaggerated or unusually flat to signal that the speaker means the opposite of what they are saying.

3. Stress: Stress is the emphasis placed on a specific syllable within a word or a specific word within a sentence. We create stress by making the sound louder, longer, and higher in pitch.

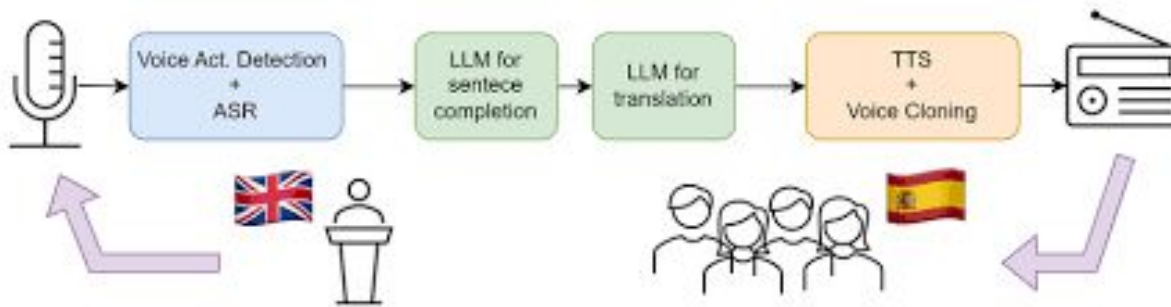
Changing which word you emphasize can completely change the meaning of the exact same sentence.

- "I didn't steal your keys." (Someone else did.)
- "I **DIDN'T** steal your keys." (Strong denial of the act.)
- "I didn't steal **YOUR** keys." (I stole someone else's keys.)

Language Identification

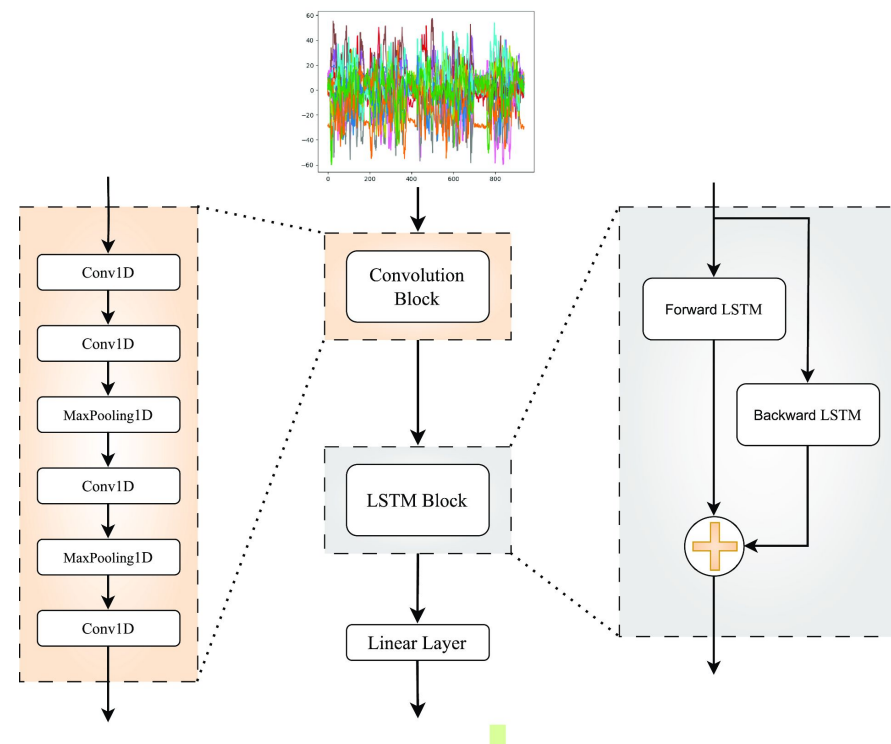
Determines the language(s) being spoken in an audio clip

- Multilingual voice assistants
- automated call routing, and
- real-time translation



Approaches

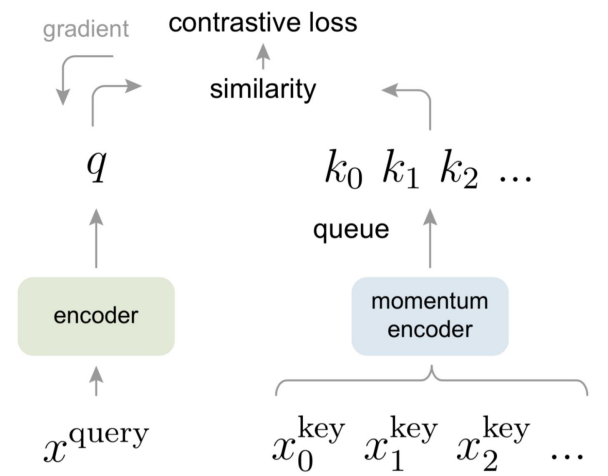
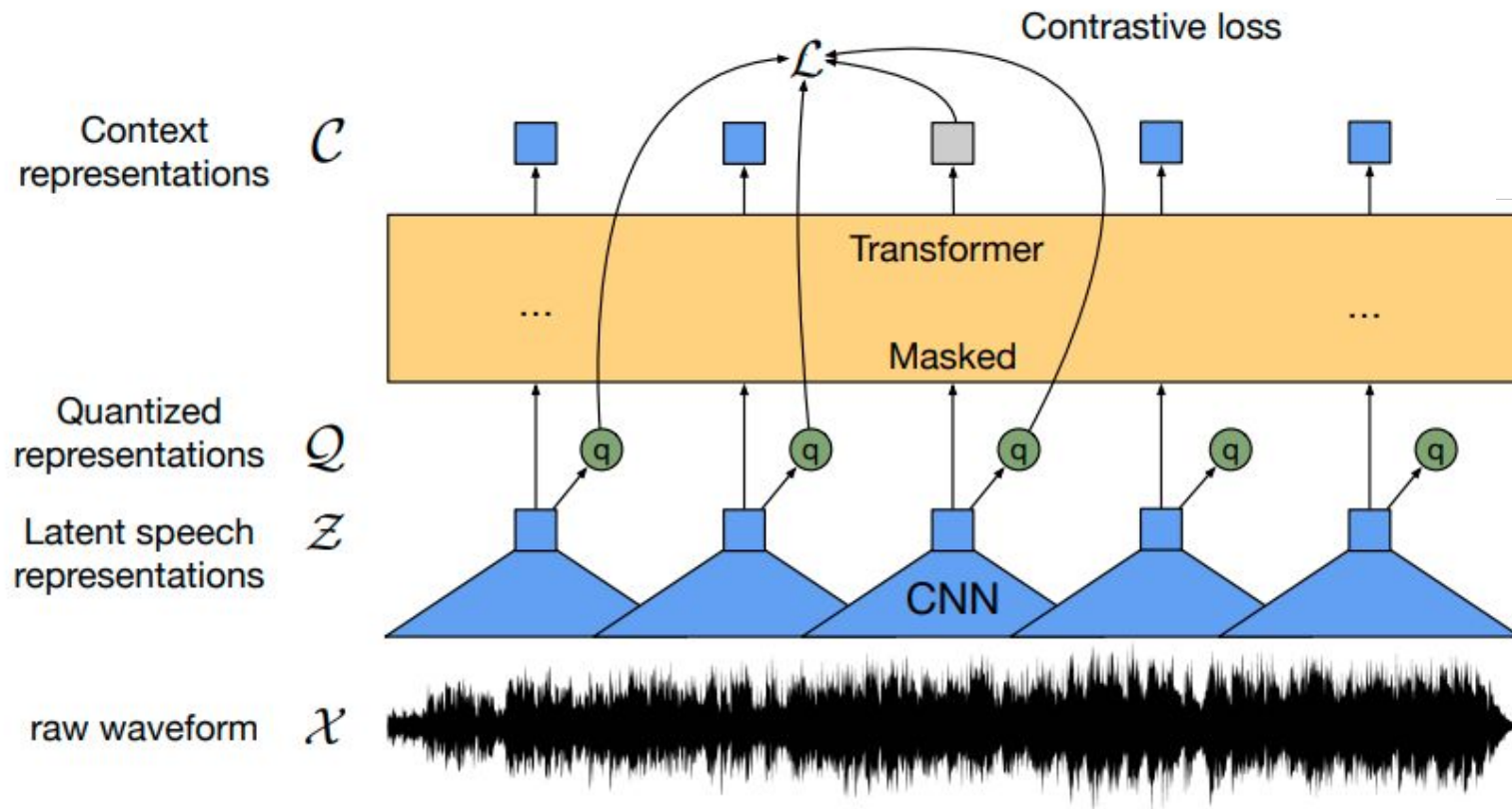
- **End-to-End Deep Learning:** Systems like CRNNs (Convolutional Recurrent Neural Networks) treat the audio like an image to find patterns.
- **Embeddings (x-vectors):** The system compresses the speech into a fixed-length mathematical vector that represents the "identity" of the language.
- **Transformers:** Using Attention mechanisms, the model focuses on specific parts of the audio (like a unique vowel sound) to decide the language.



Speech properties for LID

Feature Level	Examples
Acoustic	MFCCs, Spectrograms
Prosodic	Rhythm, Intonation, Stress
Phonotactic	Phoneme sequences (n-grams)
Lexical	Specific words/vocabulary

Self supervised learning



Metas LID system:

1. **Core Architecture:** Wav2Vec 2.0, Self-Supervised Learning (SSL).

- **Feature Encoder:** A multi-layer convolutional neural network (CNN) that takes raw audio and converts it into latent speech representations.
- **Transformer Context Network:** These representations are fed into a Transformer architecture (often the 1-billion parameter version for the 126-language model) to learn high-level context and linguistic patterns across all languages simultaneously.

2. **Training:** The massive model

- **Pre-training:** The model was initially pre-trained on roughly **5,00,000** hours of speech data across more than **1,400** languages. This allows the AI to learn the basic "building blocks" of human speech (phonemes, rhythm, and intonation) that are universal, regardless of the specific language.

3. **Fine-tuning**

To turn a general speech model into a language identifier, Meta used a process called Downstream Fine-tuning:

- **Classification Head:** The final layer of the Transformer is replaced with a linear layer that has 126 output nodes (one for each language).
- **Optimization:** The model is fine-tuned on the FLEURS (Few-shot Learning Evaluation of Universal Representations of Speech) dataset, which contains high-quality labeled audio for these 126 languages.

